tf.compat.v1.data.experimental.make_batcl Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/data/experimental/make_batched_features_dataset

> *Returns a Dataset of feature dictionaries from Example protos.*

Code Examples

```
tf.compat.v1.data.experimental.make_batched_features_dataset(
file_pattern,
batch_size,
features,
reader=None,
label_key=None,
reader_args=None,
num_epochs=None,
shuffle=True,
shuffle_buffer_size=10000,
shuffle_seed=None,
prefetch_buffer_size=None,
reader_num_threads=None,
parser_num_threads=None,
sloppy_ordering=False,
drop_final_batch=False
)
```

```
tf.compat.v1.data.experimental.make_batched_features_dataset(
file_pattern,
batch_size,
features,
reader=None,
label_key=None,
reader_args=None,
num_epochs=None,
shuffle=True,
shuffle_buffer_size=10000,
shuffle_seed=None,
prefetch_buffer_size=None,
reader_num_threads=None,
parser_num_threads=None,
sloppy_ordering=False,
drop_final_batch=False
)
```

```
serialized_examples = [
features {
feature { key: "age" value { int64_list { value: [ 0 ] } } }
feature { key: "gender" value { bytes_list { value: [ "f" ] } } }
feature { key: "kws" value { bytes_list { value: [ "code", "art" ]
} } }
},
features {
feature { key: "age" value { int64_list { value: [] } } }
feature { key: "gender" value { bytes_list { value: [ "f" ] } } }
feature { key: "kws" value { bytes_list { value: [ "sports" ] } } }
}
]
```

```
serialized_examples = [
features {
feature { key: "age" value { int64_list { value: [ 0 ] } } }
feature { key: "gender" value { bytes_list { value: [ "f" ] } } }
feature { key: "kws" value { bytes_list { value: [ "code", "art" ]
} } }
},
features {
feature { key: "age" value { int64_list { value: [] } } }
feature { key: "gender" value { bytes_list { value: [ "f" ] } } }
feature { key: "kws" value { bytes_list { value: [ "sports" ] } } }
}
]
```

```
features: {
"age": FixedLenFeature([], dtype=tf.int64, default_value=-1),
"gender": FixedLenFeature([], dtype=tf.string),
"kws": VarLenFeature(dtype=tf.string),
}
```

```
features: {
"age": FixedLenFeature([], dtype=tf.int64, default_value=-1),
"gender": FixedLenFeature([], dtype=tf.string),
"kws": VarLenFeature(dtype=tf.string),
}
```

```
{
"age": [[0], [-1]],
"gender": [["f"], ["f"]],
"kws": SparseTensor(
indices=[[0, 0], [0, 1], [1, 0]],
values=["code", "art", "sports"]
dense_shape=[2, 2]),
}
```

```
{
"age": [[0], [-1]],
"gender": [["f"], ["f"]],
"kws": SparseTensor(
indices=[[0, 0], [0, 1], [1, 0]],
values=["code", "art", "sports"]
dense_shape=[2, 2]),
}
```

Documentation

Returns a Dataset of feature dictionaries from Example protos.

If label_key argument is provided, returns a Dataset of tuple comprising of feature dictionaries and label.

Example:

We can use arguments:

And the expected output is:

Returns

Raises